University of Cape Town
Department of Computer Science

Computer Science CSC3003S
Class Test 2, 2022

Answer all questions.
Marks per question are shown in brackets.

Total Questions	**:**	**7 questions**
Total Pages (including cover)	**:**	**4 pages**

Time: 45 minutes **Marks: 35**

Section 1: Theory of Algorithms [22]

Question 1. **[8]**

a) Explain the difference between *best case*, *worst case* and *average case* time efficiency for an algorithm. [2]

b) Answer True or False to each assertion below and briefly explain your answers in each case:

i) $\Theta(\log_2 n) \subseteq \Theta(n)$ [2]

ii) if $y \in O(n)$ then $y \in O(n^2)$ [2]

iii) $n + \log n \in O(n)$ [2]

ANS:

a) The best/worst/average time efficiencies are the times (in terms of basic operations) that result when specific arrangements of *inputs* are passed into the algorithm; specifically the inputs that produce the fastest running time (best case), the slowest running time (worst case) and the average running time (average case). [at least 1 mark for noting role of INPUTS in deciding this; 2 marks if the distinction is made between best/average/worse case]

b)

i) False (1); log n has lower asymptotic growth than n, they belong to different asymptotic growth classes (big-theta is an exact bound, not upper/lower bound) (1)

ii) True (1); n is bounded above by n*n; in terms of O(.) definition, all functions in O(n) are also in O(n*n) (1)

iii) True (1); for big-oh, we can disregard terms with lower growth rates (can also use (O(max{n, log n}), which is proved in textbook) (1)

Question 2. **[6]**

Consider *ternary search* which uses the following algorithm for searching in a *sorted array* A[0..n − 1]:

If n = 1, compare the search key K with the single element of the array; otherwise, search recursively by comparing K with $A[\lfloor n/3 \rfloor]$, and if K is larger, compare it with $A[\lfloor 2n/3 \rfloor]$ to determine in which third of the array to continue the search.

i) What strategy (algorithmic pattern) does this algorithm use? [1]

ii) What is the basic operation? [1]

iii) Assuming worst-case performance, and $n = 3^k$, set up a recurrence relation (including base case) for the number of basic operations being performed and solve it using the Master Theorem (Appendix A). Show all your work. [4]

ANS:

i) decrease by constant factor (3 here) and conquer (1)

[Note: if divide and conquer is stated, award ½ mark]

ii) The basic operations is key comparison (1)

iii)

We do TWO key comparisons per stage to determine where to continue (1/ 2)

Then we recursively investigate the relevant 1/3 of the array (1/2)

For 1 element, we need to check that single element: C(1) = 1 (1/2)

Thus number of comparisons is

C(n) = 2 + C(n/3) (1)

For Master Theorem: (recall: $T(n) = aT(n/b) + f(n)$, $f(n) \in \Theta(n^d)$ and $T(1) = c$)

a = 1, b = 3, c = 1, d = 0 (1/2)

Because $a = b^d$,(1 = 3^0) ,

we use form $T(n) \in \Theta(n^d \log n)$ (1/ 2)

→ $C(n) \in \Theta(\log n)$ (1/ 2)

[Aside: In fact $C(n) = 2 \log_3 n + 1$, but the extra comparison means it loses out to binary search]

Question 3. **[4]**

Given the text *t* = RABID_RABBIT

And the pattern *p* = ABB

Construct the shift table for Horspool's algorithm for the alphabet {A-Z, _}, then run the matching algorithm, showing the steps/shifts taken when the pattern *p* is run against the text *t*.

ANS:

Table:

A	B	Other letters, _
2	1	3

½ mark for A

½ mark for B

1 mark for other alphabet characters

Apply

RABID_RABBIT

```
ABB  (fail, then 1 shift from T(B)                    1/2 mark
 ABB (fail, then 3 shifts from T(I))                  1/2 mark
    ABB  (fail, then 3 shifts from T(R) )             1/2 mark
       ABB   (match!)                                 1/2 mark
[Note: the explanations do not have to be included, but shifts
must be correct]
```

Question 4. **[4]**

Apply Warshall's algorithm to find the transitive closure of the directed graph defined by the following adjacency matrix:

$$\begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Show your working by constructing the intermediate matrices and indicating the rows/columns used for each stage.

ANS:

1 mark per stage for R1, R2, R3, R4 - 4 marks (bold shows added entries per stage); if correct rows/cols shown to compute each stage (e.g. row 1, col 1 of R(0) needed for R(1)…) but final answer is incorrect, award 1 mark at end

$$R^{(0)} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \qquad R^{(1)} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$R^{(2)} = \begin{bmatrix} 0 & 1 & \mathbf{1} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \qquad R^{(3)} = \begin{bmatrix} 0 & 1 & 1 & \mathbf{1} \\ 0 & 0 & 1 & \mathbf{1} \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$R^{(4)} = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} = T$$

Section 2: Theory of Computation [13]

Question 5. **[4]**

Briefly state the difference between:

i) An acceptable language and a decidable language [2]

ii) An upper bound and a lower bound for a problem [2]

ANS:

1 mark per "definition"

i) Acceptable: machine will answer YES & halt iff string in language.

Decidable: acceptable & machine will answer NO & halt if string not in language.

ii) Upper bound: complexity of most efficient algorithm solving it.

Lower bound: proven complexity of any algorithm solving it.

Question 6. **[5]**

Prove that the language {Turing Machines that halt when given themselves as input} is not decidable.

ANS:

Assume M^H can do this

Construct M^0 as below:

Feed input to M^H i.e. $M^H(T,T)$ where T is a TM (1)

if M^H stops with answer YES, loop forever. (1/2)
if M^H stops with answer NO, halt. (1/2)

Consider feeding M^0 as input to itself (1)

if it halts, M^H gives YES so it doesn't halt. (1/2)
if it doesn't halt, M^H gives NO so it halts. (1/2)

Contradiction. (1)

Question 7. **[4]**

Draw a transition diagram for a Turing Machine with alphabet {◊,1} that does natural number addition, and stops with the tape head at the cell immediately right of that total. A number N is represented by a string of N consecutive 1s. The tape head will start on a blank cell, followed by the numbers to add with a single blank cell between them. Example input for 2 + 4:

◊11◊1111◊

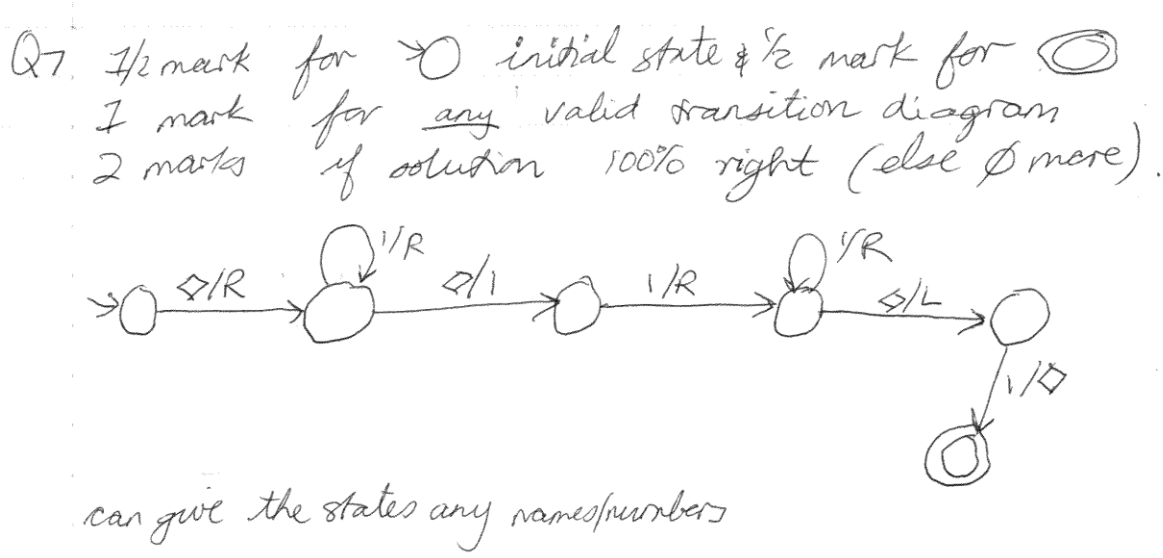
which must generate output:

◊111111◊◊

ANS:

½ mark for initial state & and ½ mark for end state
1 mark for any valid transition diagram
2 mars if solution 100% right (else no more)

Can give the states any names/numbers



Appendix A – Master Theorem

If we have a recurrence of this form: $T(n) = aT(n/b) + f(n)$, $f(n) \in \Theta(n^d)$ and $T(1) = c$ then

$T(n) \in \Theta(n^d)$ if $a < b^d$

$T(n) \in \Theta(n^d \log n)$ if $a = b^d$

$T(n) \in \Theta(n^{\log_b a})$ if $a > b^d$